UNIT-IV

THREE DIMENSIONAL VIEWING

INTRODUCTION

Graphics scenes can contain many different kinds of objects: trees, flowers, clouds, rocks, water, bricks etc. Hence, there are different methods that can be used to describe objects that will include all characteristics of these different materials. And to produce realistic displays of scenes, we need to use representations that accurately model object characteristics.

REPRESENTATION OF 3D OBJECTS

* Polygon and quadric surfaces provide precise description for simple Euclidean object such

as polyhedrons and ellipsoids.

* Spline surfaces and construction techniques are useful for designing aircraft wings, gears and other engineering structures with curved surfaces

* Octree encodings are used to represent internal features of objects, such as those obtained from medical CT images.

* Boundary representations (B-reps) describe a 3D object as a set of surfaces that separate the object interior from the environment.

* Spatial partitioning representations are used to describe interior properties by partitioning the spatial region containing an object into a set of small, nonoverlapping, continuous solids (usually cubes). A common space-partitioning description of a 3D object is an octree representation

PROJECTIONS

Once world coordinate description of the object in a scene is converted to viewing coordinate, we can project the three-dimensional object onto two dimensional view plane.

A categorical classification of different types of projections commonly taken into consideration is shown below in fig 1.

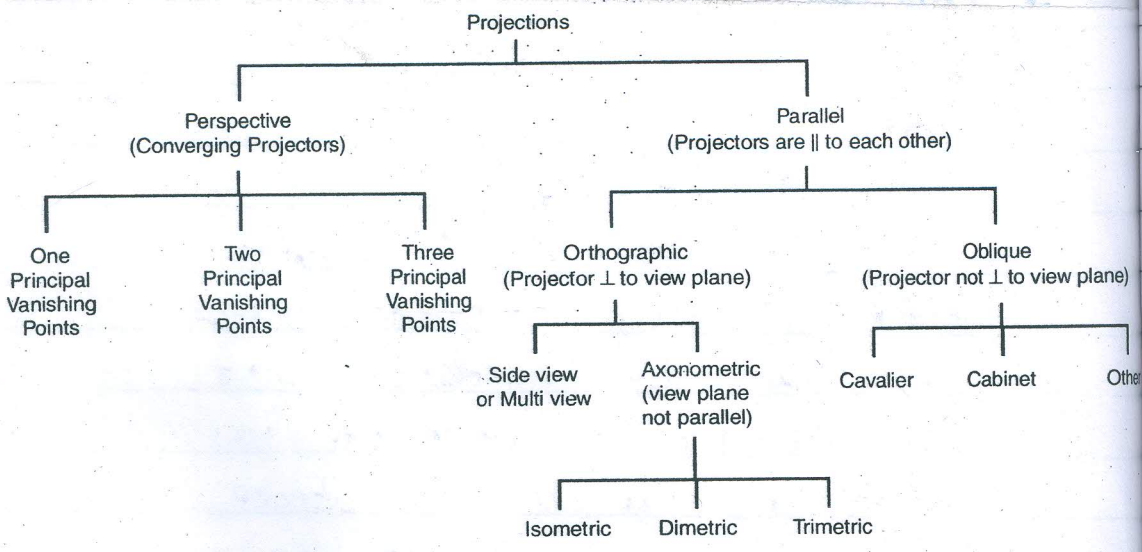

Fig: 1 *Classification of projection schemes.*

There are two basic projection methods
1. Parallel projection
2. Perspective projection.

Parallel Projection - In parallel projection coordinate positions are transformed to the viewplane along parallel lines as shown in fig 2.

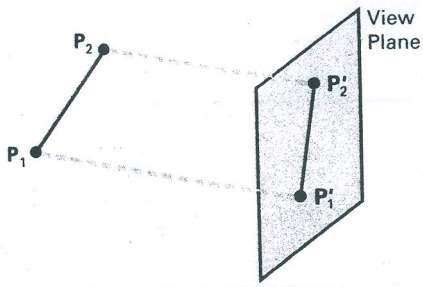

Fig : 2:
Parallel projection of an object to the view plane.

Parallel projections preserves the relative proportions of object, and this is the method used in drafting to produce scale drawings of 3D objects. Accurate views of various sides of an object are obtained with parallel projection, but this does not give a realistic representation of the appearance of a 3D object.

Perspective Projection - For this projection, object positions are transformed to the viewplane along lines that converge to a point called the projection reference point or center of projection. The projected view of an object is determined by calculating the intersection of the projected lines with the view plane as shown in fig 3.

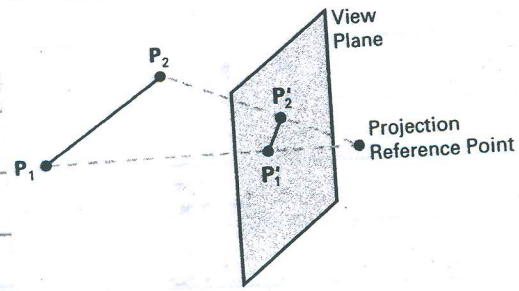

Fig: 3
Perspective projection of an object to the view plane.

A perspective projection, produces realistic views but does not preserve relative proportions. Projections of distant objects are smaller than projection of objects of the same size that are closer to projection plane as shown in fig 4.

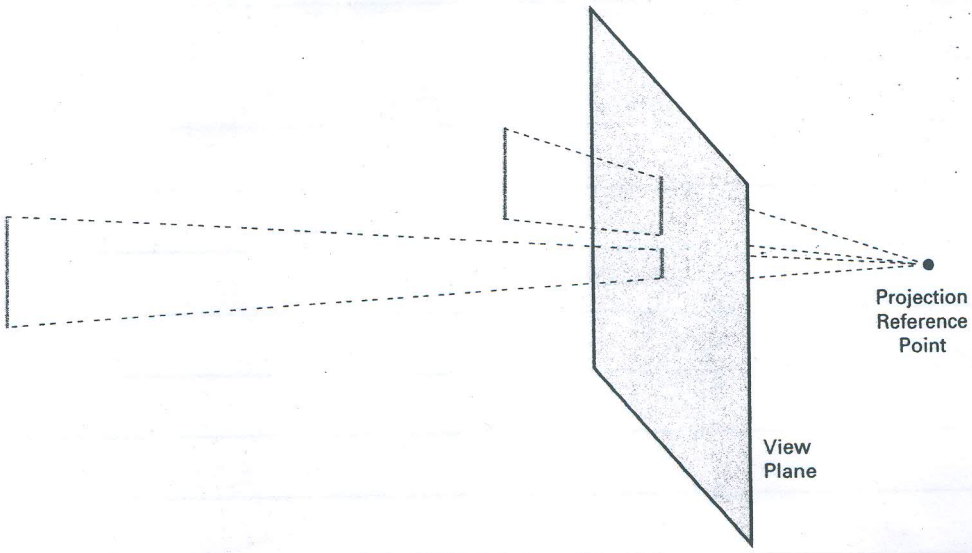

Fig 5.4:
Perspective projection of equal-sized objects at different distances from the view plane.

PARALLEL PROJECTIONS

We can specify a parallel projection with a projection vector that defines the direction for projection lines.

Types of parallel projections:
1) Orthographic projection
2) Oblique parallel projection.

Orthographic Parallel Projection - when the projection is perpendicular to the view plane.

Oblique Parallel Projection - When the projection is not perpendicular to the view plane.

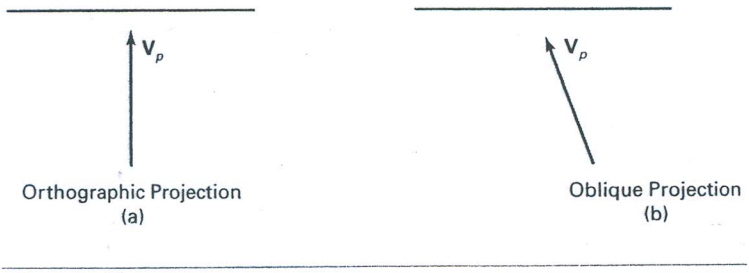

Fig : 5
Orientation of the projection vector V_p to produce an orthographic projection (a) and an oblique projection (b).

The above fig 5 depicts the two types of paralled projection.

ORTHOGRAPHIC PARALLEL PROJECTION

Orthographic projections are most often used to produce the front, side and top views of an object. as shown in fig 6

front, side and rear orthographic projection of an object are called elevations and top Orthographic projection is is called a plan view.

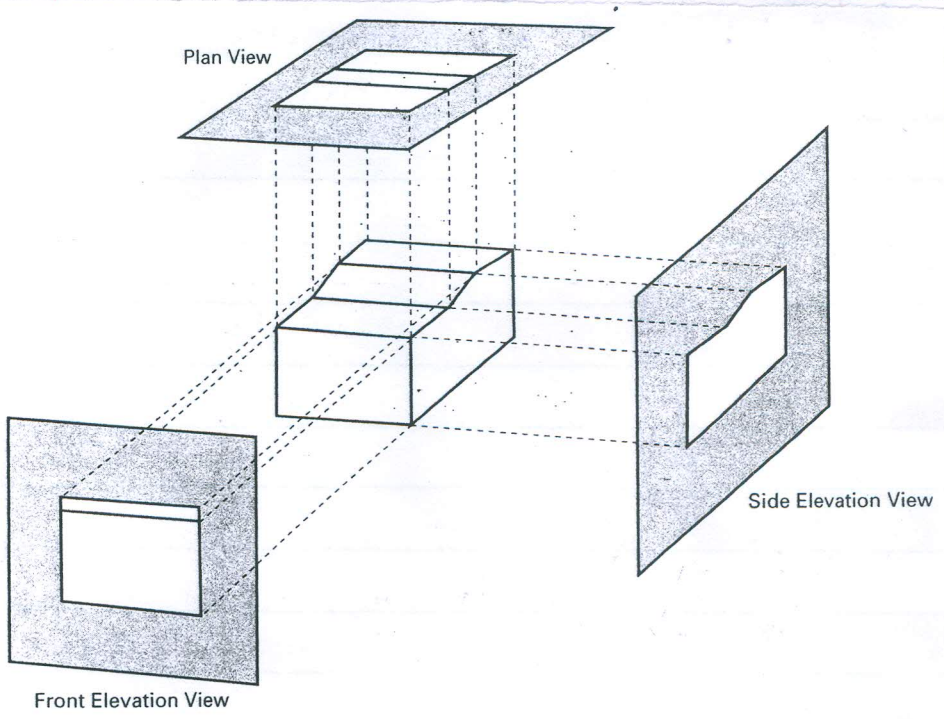

Fig: 6
Orthographic projections of an object, displaying plan and elevation views.

We can also form orthographic projection that display more than one face of an object. Such views are called axometric othographic projections. The most commonly used axonometric projection is the isometric projection.

We generate an isometric projection by aligning the projection plane so that it intersects with each coordinate axis in which the object is defined (called the

principal axes) at the same distance from origin.

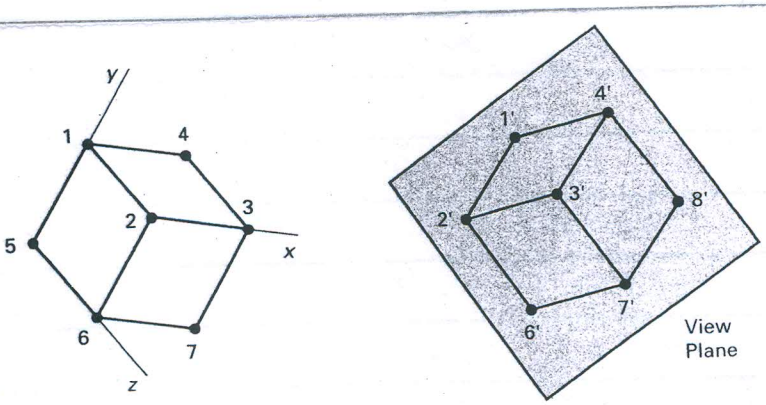

Fig 7.
Isometric projection for a cube.

Fig 7 shows an isometric projection for a cube. It is obtained by aligning the projection vector with the cube diagonal.

All the three principal axis are foreshortened equally in an isometric projection so that relative proportions are maintained.

Transformation equations for an orthographic parallel projection are straightforward.

If a viewplane is placed at position z_{vp} along the z_v axis as shown in fig 8,

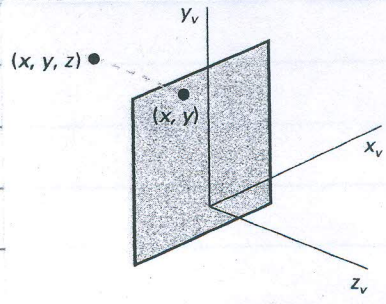

Fig: 8.
Orthographic projection of a point onto a viewing plane.

then any point (x, y, z) in viewing coordinates is transformed to projection coordinates as

$$x_p = x$$
$$y_p = y \quad \text{—— (1)}$$

where the original z coordinate value is preserved for the depth information needed in depth cueing and visible surface determination procedures

OBLIQUE PARALLEL PROJECTION

An oblique projection is obtained by projecting points along parallel

lines that are not perpendicular to the projection plane. An oblique projection vector is specified with two angles α and ϕ as shown in fig 9.

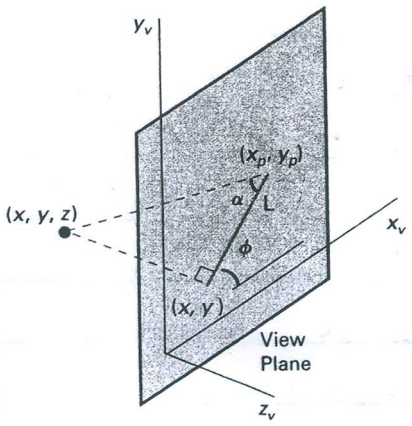

Fig: 9
Oblique projection of coordinate position (x, y, z) to position (x_p, y_p) on the view plane.

Point (x, y, z) is projected to position (x_p, y_p) on the view plane. Orthographic projection coordinates on the plane are (x, y). The oblique projection line from (x, y, z) to (x_p, y_p) makes an angle α with the line on the projection plane the joins (x_p, y_p) and (x, y).

This line, of length L, is at angle ϕ with the horizontal direction in the projected plane.

We can express the projection coordinates in terms of x, y, L and ϕ as

$$x_p = x + L\cos\phi \quad \text{— (2)}$$

$$y_p = y + L\sin\phi$$

Length L depends on the angle α and the z coordinate of the point to be projected:

$$\tan\alpha = \frac{z}{L} \quad \text{— (3)}$$

Thus,

$$L = \frac{z}{\tan\alpha} \quad \text{— (4)}$$

$$= zL_1$$

where L_1 is the inverse of $\tan\alpha$, which is also the value of L when $z = 1$.

We can write the oblique projection equation (2) as

$$x_p = x + z(L_1\cos\phi) \quad \text{— (5)}$$
$$y_p = y + z(L_1\sin\phi)$$

The transformation matrix for producing any parallel projection onto the $x_v y_v$ plane can be written as

$$M_{parallel} = \begin{bmatrix} 1 & 0 & L_1 \cos\phi & 0 \\ 0 & 1 & L_1 \sin\phi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

An orthographic projection is obtained when $L_1 = 0$ (which occurs at a projection angle α of $90°$)

Oblique projections are generated with nonzero values for L_1.

CAVALIER PARALLEL PROJECTION

In this, the lines perpendicular to projection plane are preserved in length, i.e $L_1 = 1$. In this case $\alpha = 45°$ so that $\tan\alpha = 1$.

Examples of cavalier projection for a cube are given in Fig 10.

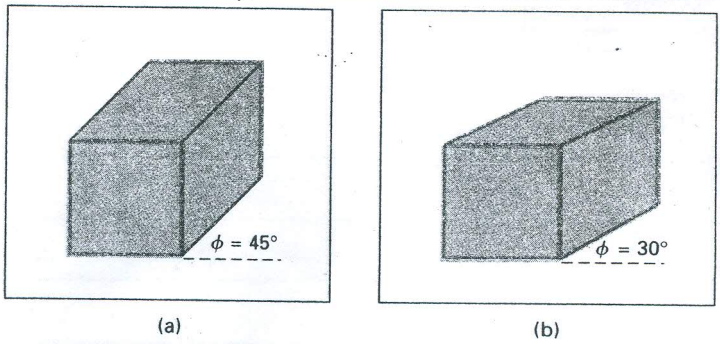

Fig: 10
Cavalier projections of a cube onto a view plane for two values of angle ϕ.
Note: Depth of the cube is projected equal to the width and height.

CABINET PARALLEL PROJECTIONS

In this, lines perpendicular to projection plane are 1/2 of their true length, i.e. $L_1 = 1/2$.
When the projection angle α is chosen so that $\tan\alpha = 2$, the resulting view is called a cabinet projection.
Fig 11 shows examples of cabinet projection for a cube.

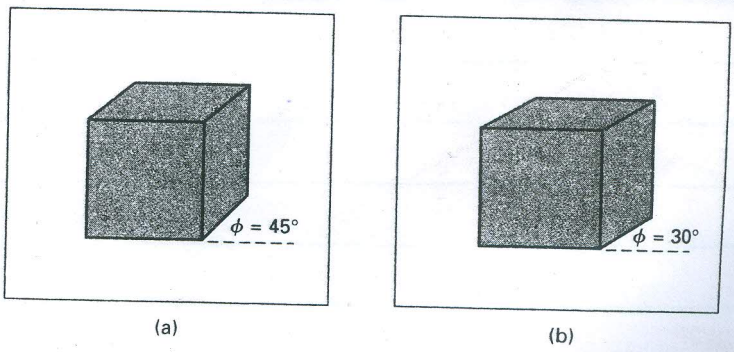

Fig: 11
Cabinet projections of a cube onto a view plane for two values of angle ϕ. Depth is projected as one-half that of the width and height.

PERSPECTIVE PROJECTIONS

In a perspective projection, the object is very far from the viewer and it seems very small.

In such projection, the lines of projection are not parallel. Instead, they all converge at a single point called the centre of projection. The projection gives the image which would be seen if the viewer's eye were located at the centre of projection.

Suppose we set the projection reference point at position z_{prp} along the z_v axis, and we place the viewplane at z_{vp} as shown in fig 12

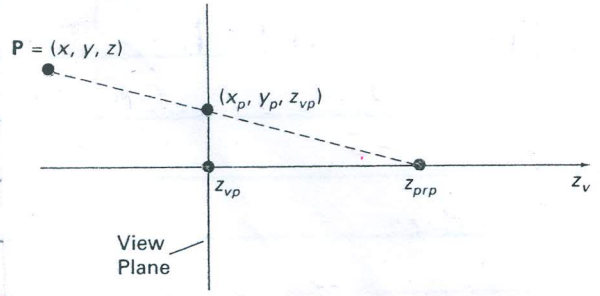

Fig: 12
Perspective projection of a point **P** with coordinates (x, y, z) to position (x_p, y_p, z_{vp}) on the view plane.

We can write equations describing coordinate positions along this perspective projection line in parametric form as:

$$x' = x - xu$$
$$y' = y - yu$$
$$z' = z - (z - z_{prp})u$$

Parameter u takes values from 0 to 1 and coordinate position (x', y', z') represent any point along the projection line.

When $u = 0$, we are at position $P = (x, y, z)$

When $u = 1$, we have projection reference point coordinates $(0, 0, z_{prp})$.

On the viewplane $z' = z_{vp}$ and we can solve the z' equation for parameter u at this position along projection line:

$$u = \frac{z_{vp} - z}{z_{prp} - z}$$

Substituting this value of u into equations for x' and y', we obtain perspective transformation equations

$$x_p = x\left(\frac{z_{prp} - z_{vp}}{z_{prp} - z}\right) = x\left(\frac{d_p}{z_{prp} - z}\right)$$

$$y_p = y\left(\frac{z_{prp} - z_{vp}}{z_{prp} - z}\right) = y\left(\frac{d_p}{z_{prp} - z}\right)$$

where $d_p = z_{prp} - z_{vp}$ is the distance of viewplane from the projection reference point.

Using a 3D homogeneous-coordinate representation, we can write the above perspective projection transformation in matrix form as:

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -z_{vp}/d_p & z_{vp}(z_{prp}/d_p) \\ 0 & 0 & -1/d_p & z_{prp}/d_p \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

In this representation, the homogeneous factor is

$$h = \frac{z_{prp} - z}{d_p}$$

and the projection coordinates on the view plane are calculated from the homogeneous coordinates as:

$$x_p = \frac{x_h}{h}, \quad y_p = \frac{y_h}{h}.$$

where the original z-coordinate value would be retained in projection coordinates for visible surface and other depth processing.

When a 3D object is projected onto a viewplane using perspective projection equations, any set of parallel lines in the object that are not parallel to the plane are projected into converging lines.

The point at which a set of projected parallel lines appears to converge is called a <u>vanishing point</u>.

The vanishing point for any set of lines that are parallel to one of the principal axes of an object is referred to as principal vanishing point.

We control the no. of principal vanishing points (one, two or three) with the orientation of projection plane.

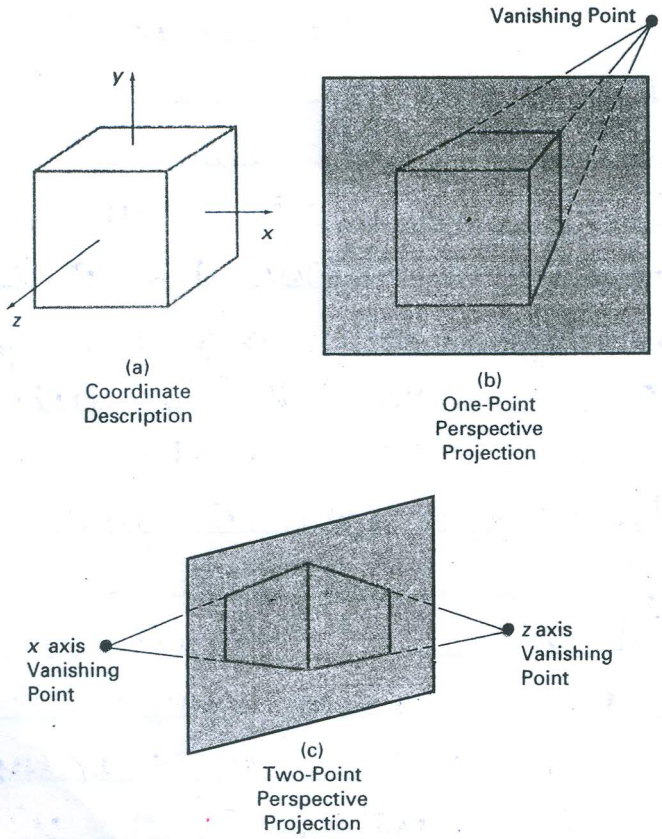

Fig: 13.
Perspective views and principal vanishing points of a cube for various orientations of the view plane relative to the principal axes of the object.

The perspective projection are accordingly classified as one-point, two-point or three-point projection.

The no. of vanishing points i.e principal vanishing points in a projection is determined by the no. of principal axes intersecting the view plane.

Fig 13 illustrates the appearance of one point and two point perspective projection.

In fig 13(b), view plane is aligned parallel to the xy object plane so that only the object z axis is intersected. This orientation produces a one-point perspective projection with a z-axis vanishing point.

Fig 13(c), the projection plane intersects both the x and z axes but not the y axis. The resulting two point perspective projection contains both x axis and z axis vanishing points.

Let us see the differences between parallel and perspective projections–

Parallel Projection	Perspective Projection
1. In parallel projection, the centre of projection **is at infinity**.	1. In perspective projection, the centre of projection, is at a **finite distance**
2. Here, all projectors are parallel to each other.	2. Here, projectors are not parallel.
3. It is a less realistic view.	3. It is more realistic.
4. It can be used for the applications where exact measurement is required.	4. It resembles to that of our photographic systems (camera) and human eye.
5. These are linear transforms implemented with a matrix.	5. These are nonlinear transforms.
Examples: Useful for drawing schematic diagrams.	**Examples:** Useful in architectural rendering, realistic views.

THREE DIMENSIONAL CLIPPING

2D clipping algorithms can be extended for handling 3D situations. Hence, instead of clipping boundaries, we use clipping planes and clip against a clipping volume.

In 2D, we consider four boundaries of the clipping window while in 3D we consider six surfaces.

3D COHEN-SUTHERLAND CLIPPING ALGORITHM

Again, Cohen-Sutherland algorithm is found to be most popular because of its simplicity.

A surface is considered to be the clipping window. It can be uniquely specified by its six parameters i.e x_{min}, x_{max}, y_{min}, y_{max}, z_{min} and z_{max}.

It uses an outcode of 6 bits

Visibility types in 3D

There are three visibility types in 3D

1) Totally visible
2) Totally invisible
3) Partially visible (Clipping candidate)

* Each endpoint is assigned a 6 bit region code or outcode.

a bit is true (1) when appropriate condition is satisfied:

			True	False
BIT 1	point is above view volume	$y <= y_{max}$	0	1
BIT 2	point is below view volume	$y >= y_{min}$	0	1
BIT 3	point is right view volume	$x <= x_{max}$	0	1
BIT 4	point is left of view volume	$x >= x_{min}$	0	1
BIT 5	point is behind view volume	$z <= z_{max}$	0	1
BIT 6	point is in front of view volume	$z >= z_{min}$	0	1

In fig 14, region codes for all points are shown.

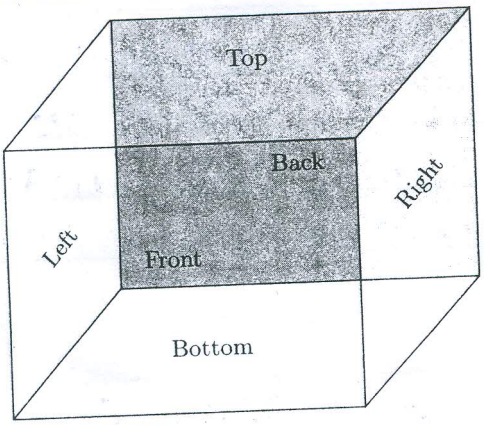

Fig 14 : 3-D clipping window.

* Once region codes for all points is determined, we can easily identify the visibility of a point.

* A line segment is totally visible i.e inside the clipping window if both the region codes are 000000.

* If the logical AND of two endpoint codes is not 000000, then the line falls in category 2 i.e it is totally invisible.

* If the logical AND of two endpoints codes is 000000 then the line falls in category 3 i.e the clipping candidate as it is partially visible and hence compute the intersection with the boundary planes.

Computing intersection points

* The intersection calculations use the parametric representation of a line from $P_0(x_0, y_0, z_0)$ to $P_1(x_1, y_1, z_1)$

$$x = x_0 + t(x_1 - x_0)$$
$$y = y_0 + t(y_1 - y_0)$$
$$z = z_0 + t(z_1 - z_0)$$
$$0 \le t \le 1$$

As t varies from 0 to 1, the three equations give the coordinates of all points on the line, from P_0 to P_1.

* So for $y = 1$, replace y with 1 and solve for t.
* If $0 \le t \le 1$ use it to find x & z.
* Test if x and z are in valid range.
* Repeat for planes $x = 1$, $z = 1$.

Computing the intersection

* To clip against the face, above compute

$$t = \frac{(y_{max} - y_0)}{(y_1 - y_0)}$$

and use it to evaluate the x & z intersection. The value of y coordinate is $y = y_{max}$.

* To clip against the face, Below, compute

$$t = \frac{(y_{min} - y_0)}{(y_1 - y_0)}$$

and use it to evaluate the x & z intersection. The value of y coordinate is $y = y_{min}$.

* To clip against the face, Right, compute

$$t = \frac{(x_{max} - x_0)}{(x_1 - x_0)}$$

and use it to evaluate the y & z intersection. The value of x coordinate $x = x_{max}$.

* To clip against a face, Left, compute

$$t = \frac{(x_{min} - x_0)}{(x_1 - x_0)}$$

and use it to evaluate the y & z intersection. The value of x coordinate $x = x_{min}$.

* To clip against a face, Behind
compute

$$t = \frac{(z_{max} - z_0)}{(z_1 - z_0)}$$

and use it to evaluate the x & y intersection. The value of z coordinate $z = z_{max}$.

* To clip against a face, Front
compute

$$t = \frac{(z_{min} - z_0)}{(z_1 - z_0)}$$

and use it to evaluate x & y intersection. The value of z coordinate $z = z_{min}$.

HIDDEN SURFACE REMOVAL

In a given set of 3D objects and viewing specification we wish to determine which lines or surfaces of the object are visible so that we can display only the visible lines or surfaces. This process is known as hidden surface elimination or visible surface detection.

The hidden line or surface algorithms determine the lines, edges, surfaces or volumes that are visible or invisible to an observer located at a specific point in space.

These algorithms are broadly classified according to whether they deal with object definition directly or with their projected images. These two approaches are:

1) Object space Methods
2) Image space methods

Object Space Method

This method is implemented in the physical coordinate system in which objects are described. It compares objects and parts objects to each other within the scene definition to determine which surfaces, as a whole we should label as visible. Object-space methods are generally used in line-display algorithms.

Image Space Method

This method is implemented in the screen coordinate system in which the objects are viewed in an image space algorithm, visibility is decided point by point at each pixel position on the view plane. Most visible surface algorithms use image space methods, although object space methods can be used effectively to locate visible surfaces in some cases.

DEPTH BUFFER (z-BUFFER) METHOD

A commonly used image space approach to detecting visible surfaces is the depth-buffer method, which compares surface depths at each pixel position on projection plane. This procedure is also referred to as the z-buffer method, since object depth is usually measured from the viewplane along the z-axis of a viewing system.

When object description is converted to projection coordinates each (x, y, z) pixel position on view plane is specified by (x, y) coordinate and z value gives the depth information. Fig 15 shows three surfaces at varying distances along orthographic projection line from position (x, y) in a view plane taken as (x_v, y_v) plane. Surface S_1 is closest at this position, so

its surface intensity value at (x, y) is saved.

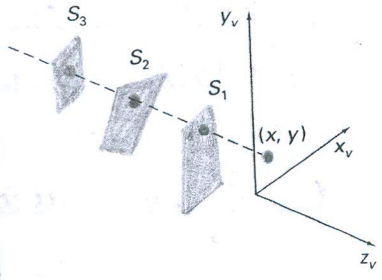

Fig: 15
At view-plane position (x, y), surface S_1 has the smallest depth from the view plane and so is visible at that position.

The z-buffer algorithm is usually implemented in the normalized coordinates, so that z values range from 0 at the back clipping plane to z_{max} at the front clipping plane. (The value of z_{max} can be set either to 1 (for unit cube) or to the largest value that can be stored on the system).

The implementation requires two buffer areas: A depth buffer or (z-buffer) is used to store depth values for each (x, y) position as surfaces are processed and the refresh buffer stores the intensity values for each position.

Initially, all positions in the depth buffer (z-buffer) are set to 0 (minimum depth) and refresh buffer is initialized to the background intensity. Each surface listed in polygon tables is then processed one scan line at a time, calculating the depth (z value) at each (x,y) pixel position.

The calculated depth is compared to the value previously stored in depth buffer at that position. If calculated depth is greater than the value stored in the depth buffer the new depth value is stored, and the surface intensity at that position is determined and placed in the same xy location in the refresh buffer.

For eg in fig 15, among three surfaces, surface S, has smallest depth at view position (x,y) and hence highest z-value. Hence, it is visible at that position.

DEPTH BUFFER (Z-BUFFER) ALGORITHM

STEP 1: Initialize the depth buffer and refresh buffer so that for all buffer positions (x, y),

$$depth(x, y) = 0 \quad , \quad refresh(x, y) = I_{backgnd}$$

STEP 2: For each position on each polygon surface, compare depth values to previously stored values in depth buffer to determine visibility.

* Calculate the depth z for each (x, y) position on the polygon
* If $z > depth(x, y)$, then set

$$depth(x, y) = z \quad , \quad refresh(x, y) = I_{surf}(x, y)$$

where $I_{backgnd}$ is the value for the background intensity and $I_{surf}(x, y)$ is the projected intensity value for the surface at pixel position (x, y). After all surfaces have been processed, depth buffer contain depth values for visible surfaces & refresh buffer contains the corresponding intensity values for those surfaces.

To calculate depth values (z-values) for a surface position (x, y) the plane equation

$$Ax + By + Cz + D = 0 \quad -(1)$$

is used, where (x, y, z) is any point on plane. and A, B, C & D are constants

Therefore, we can write

$$z = \frac{-Ax - By - D}{C} \quad -(2)$$

For any scan line (Fig 16), adjacent horizontal position across the line differ by 1, and vertical y value on an adjacent scan line differs by 1.

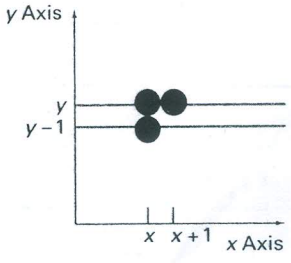

Fig: 16
From position (x, y) on a scan line, the next position across the line has coordinates $(x + 1, y)$, and the position immediately below on the next line has coordinates $(x, y - 1)$.

If the depth of position (x, y) has been determined to be z, then the depth z' of next position $(x+1, y)$ along the scan line is obtained from eq (2) as

$$z' = -\frac{A(x+1) - By - D}{C} \quad -(3)$$

$$\boxed{z' = z - \frac{A}{C}} \quad -(4)$$

The ratio $-A/C$ is constant for each surface, so succeeding depth values across a scan line are obtained from preceeding values with a single addition.

A similar incremental calculation can be performed to determine the first value of z on the next scan line, decrementing B/C for each $(y-1)$.

Advantages

1) It is simple to use and easy to implement.
2) It displays complex intersections easily
3) Any kind of surfaces can be handled
4) No depth storing of objects is needed and hence computational complexity is linear.

Disadvantages

1) Storage requirement is higher
2) Since depth storing is not done, a location in the z-buffer may have to be changed many things depending upon the no. of surfaces representing the scene.
3) It is time consuming process as it requires comparison for each pixel instead of for the entire polygon.

DEPTH - SORTING METHOD (PAINTER'S ALGORITHM)

Using both image-space and object space operations, the depth sorting method performs the following basic functions:

1) Surfaces are sorted in order of decreasing depth.
2) Surfaces are scan converted in order, starting with the surface of greatest depth.

* Sorting operations are carried in both image and object space
* Scan conversion of the polygon surface is performed in image space.

This method of solving the hidden-surface problem is often referred to as painter's algorithm.

In creating oil painting, an artist first paints the background colors. Next, the most distant

objects are added, then the nearer objects and so forth. At the final step, the foreground objects are painted on the canvas over background and other objects that have been painted on the canvas. Each layer of paint covers up the previous layer.

Using a similar technique, we first sort surfaces according to their distance from the view plane. The intensity values for the farthest surface are then entered into the refresh buffer. Taking each succeeding surface in turn (in decreasing depth order), we "paint" the surface intensities onto the frame buffer over the intensities of previously processed surfaces.

Painting polygon surfaces onto the frame buffer according to depth is carried out in several steps. Assuming we are viewing along -z direction, surfaces are ordered on the first pass according to

the smallest z value on each surface. Surface S with the greatest depth is then compared to the other surface in the list to determine whether there are any overlaps in depth. If no overlap occur, S is scan converted.
Fig 17 shows two surfaces that overlap in xy plane but have no depth overlap.

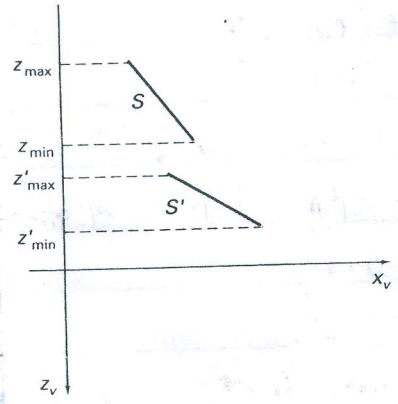

Fig: 17
Two surfaces with no depth overlap.

This process is then repeated for the next surface in the list. As long as no overlaps occur, each surface is processed in depth order until all have been scan converted.

If a depth overlap is detected at any point in the list we need to make some additional comparisons to determine whether any of the surfaces should be reordered.
We make the following tests for each surface that overlaps with S. If any one of these tests is true, no reordering is necessary for that surface. The tests are listed in order of increasing difficulty.

1. The boundary rectangles in the xy plane for the two surfaces do not overlap.

2. Surface S is completely behind the overlapping surface relative to the viewing position.

3. The overlapping surface is completely in front of S relative to the viewing position.

4. The projections of the two surfaces

onto the view plane do not overlap.

We perform these tests in the order listed and proceed to the next overlapping surface as soon as we find one of the test is true.

If all the overlapping surfaces pass atleast one of these tests, none of them is behind S. No reordering is then necessary and S is scan converted.

Test 1 is performed in two parts. We first check for overlap in the x direction, then we check for overlap in the y direction. If either of these directions show no overlap, the two planes cannot obscure one other. An example of two surfaces that overlap in the z direction but not in the x direction as shown in Fig 18.

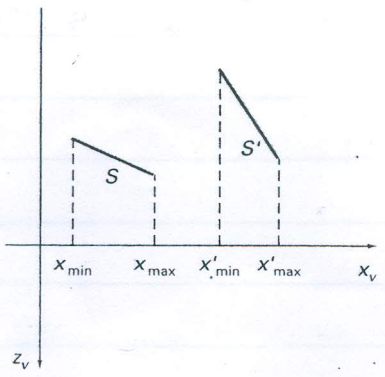

Fig: 18
Two surfaces with depth overlap but no overlap in the x direction.

We can perform tests 2 and 3 with an "inside-outside" polygon test. That is, we substitute the coordinates for all vertices of S into the plane equation for the overlapping surface and check the sign of the result. If the plane equations are set up so that the outside of the surface is toward the viewing position, then S is behind S' if all vertices of S are "inside" S' (Fig 19). Similarly S' is completely in front of S if all vertices of S are "outside" of S'. Fig 20 shows an overlapping surface S' that is completely in front of S, but surface S is not completely "inside" S' (test 2 is not true).

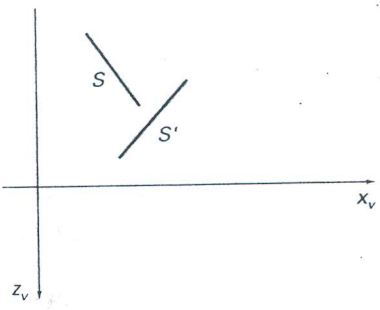

Fig:19
Surface S is completely behind ("inside") the overlapping surface S'.

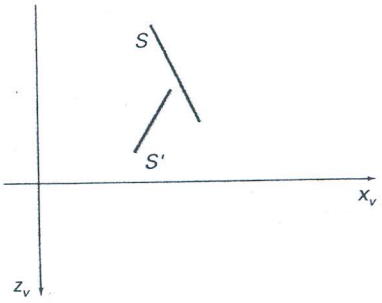

Fig:20
Overlapping surface S' is completely in front ("outside") of surface S, but S is not completely behind S'.

If tests 1 through 3 have all failed, we try test 4 by checking for intersection between the bounding edges of the two surfaces using line equations in the xy plane. As demonstrated in fig 21, two surfaces may or may not intersect even though their coordinate extends overlap in the x, y and z directions.

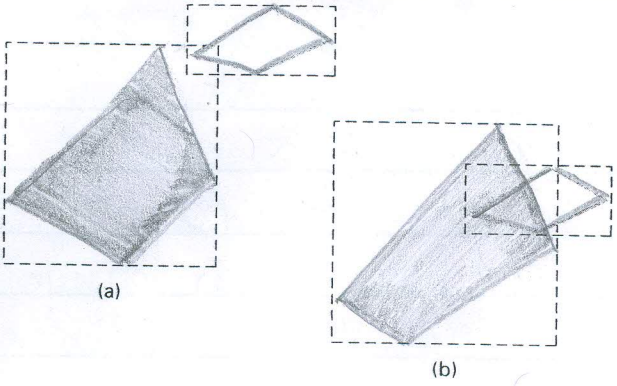

fig: 21
Two surfaces with overlapping bounding rectangles in the *xy* plane.

Should all four tests fail with a particular overlapping surface S', we interchange surfaces S and S' in the sorted list. An example of two surfaces that would be reordered with this procedure is given in fig 22

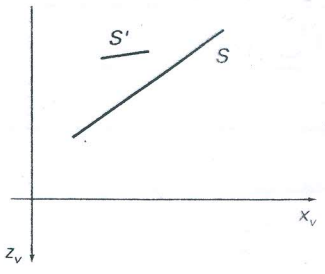

fig: 22
Surface *S* has greater depth but obscures surface *S'*.

Fig 23 illustrates a situation in which we would first interchange S and S''. But since S'' obscure part of S', we need to interchange S'' and S' to get the three surfaces into the correct depth order.

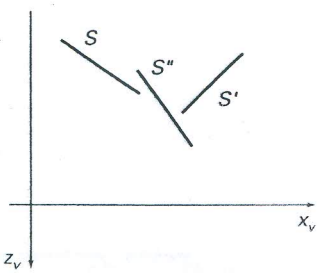

Fig: 23
Three surfaces entered into the sorted surface list in the order S, S', S'' should be reordered S', S'', S.

Therefore, we need to repeat the testing process for each surface that is reordered in the last.